

OOP C++ Assignment - I

Name:- K. Shashank

Roll No. :- 21071A12F2

Class:- II Year I Sem

Sec:- IT-C

1) Q) Explain the characteristics of Object Oriented programming language.

Ans) The characteristics of oop are :-

1. Object :- objects are basic units of oop. They are instances of class which have data members and uses various member functions to perform tasks.
2. Class :- It is similar to structures in 'C' language. class can also be defined as user defined data type but it also contains functions in it. so class is basically a blueprint for object. It declares and defines what data variables the object will have and what operations can be performed on the object.
3. Abstraction :- Abstraction refers to showing only the essential features of an application and hiding the details. In c++ class can provide methods vary to outside world to access and use the data variable. Keeping the variables hidden from direct access or classes even declare everything accesible to everyone or just to classes inserts inheriting it. This can be done using access specifiers.
4. Encapsulation :- It is also be called as data binding. Encapsulation is all about binding the data variables and functions in class.
5. Inheritance :- It is the process of objects of one class aguring the properties of objects of another class. The main advantage of inheritance is to provide the data reusability i.e, we can add additional features to the existing class without modifying it.

6. Polymorphism: It refers to the ability to take more than one form. In polymorphism an operation can exhibit different behaviours in different instances. For example consider the operation of addition of two numbers then operation will generate a sum. If the operands are strings then the operation would produce a third string by concatenation.

7. Overloading: It is one of the branches of Polymorphism. If an existing operator or function is applied with new datatype. It is called as Overloading.

→ Overloading is again categorized into :-

- i) operator overloading
- ii) Function overloading.

8. Exceptional Handling: It is used to handle unresolved exceptions or errors produced at runtime.

Q/2) Write about constructors and Destructors in detail with relevant example.

Ans!

A constructor is a special function, which is used to initialize the object members.

- constructor is invoked (or) called automatically by the compiler whenever the object is created.
- A constructor will have the same name as that of the class.
- constructors can be defined either inside the class or outside the class, using membership label.
- constructors are of 3 types. They are:-
 - i) Default constructor
 - ii) Parametrised constructor
 - iii) Copy constructor

i) Default constructor:-

A constructor which doesnot receive any parameters is called as Default constructor or Zero Argument constructor. Every class object is initialized with the same set of values in the default constructor.

Even if a constructor is not defined explicitly, the compiler will provide a default constructor implicitly.

Ex:- Syntax:-

```
class A
{
    ≡
    public
    {
        A()
    }
    ~A()
    {
    }
}
```

↓
declaration

↓
to free space

Ex:-

```
class cube
{
    public:
    int side;
    cube()
    {
        side = 10;
    }
};

int main()
{
    cube c;
    cout << c.side;
}
```

Output :-

10

```
class cube
{
    public:
    int side;
};

int main()
{
    cube c;
    cout << c.side;
}
```

Output :-

0 or any random value

Qii) Parameterized Constructor:

Using default constructor, it is impossible to initialize different objects with different initial values. There is a need of parameterized constructor if we want to pass arguments to constructors which are needed to initialize an object.

A Parameterized Constructor is a constructor that can accept one or more arguments. This helps programmers assign varied initial values to an object at the creation time.

Syntax:-

```
classname (Arg 1, Arg 2, ...)
{
    data member 1 = Arg 1;
    data member 2 = Arg 2;
    !
}
```

Ex}

```
class cube
{
    public:
    int side;

    cube (int x)
    {
        side = x;
    }
};
```

```
int main()
{
    cube c1 (10);
    cube c2 (20);
    cube c3 (30);
    cout << c1.side;
    cout << c2.side;
    cout << c3.side;
}
```

Output:-

10
20
30

iii) Copy constructor!

A copy constructor is a type of constructor used to create a copy of an already existing object of a class type. The compiler provides a default copy constructor to all the classes. A copy constructor comes into the picture whenever there is a need for an object with the same values for data members as an already existing object. A copy constructor is invoked when an existing object is passed as a parameter.

```
Ex: #include <iostream>
using namespace std;
class wall {
    private:
        double length;
        double height;
    public:
        wall (double len, double hgt)
        {
            length = len;
            height = hgt;
        }
        wall (wall &obj)
        {
            length = obj.length;
            height = obj.height;
        }
        double calculateArea()
        {
            return length * height;
        }
};

int main()
{
    wall w1(10.5, 8.6);
    cout << "Area of Room 1: " << w1.calculateArea() << endl;
    wall w2 = w1;
    cout << "Area of Room 2: " << w2.calculateArea() << endl;
    return 0;
}
```

Output:

Area of Room 1 : 90.3
Area of Room 2 : 90.3

Destructors in C++ :

Destructor is a special class function which destroys the object as soon as the scope of object ends. The destructor is called automatically by the compiler when the object goes out of scope.

The syntax for destructor is same as that for the constructor, the class name is used for the name of destructor, with a tilde ~ sign as prefix to it.

Syntax :-

```
class A
{
    public:
        // defining destructor for class
        ~A()
        {
            // statement
        }
};
```

* Destructors will never have any arguments.

Ex!

```
class A
{
    // constructor
    A()
    {
        cout << "constructor called ";
    }
    // destructor
    ~A()
    {
        cout << "Destructor called ";
    }
}

int main() {
    A obj1; // constructor called
    int x = 1;
    if (x)
    {
        A obj2; // constructor called
    }
}
```

output :

Constructor called
constructor called
Destructor called
Destructor called.

Q) 3) Discuss about friend functions and friend classes with example programmes.

Ans: Friend Function in C++ :

A friend function can access the private and protected data of a class. We declare a friend function using the 'friend' keyword inside the body of the class.

Syntax :-

```
class class_name {  
    .....  
    friend returnType functionName (arguments) ;  
    .....  
}
```

// C++ program to demonstrate the ^{working} ~~following~~ of friend function.

```
#include <iostream>  
using namespace std;  
class Distance {  
    private:  
        int meter;  
        // friend function  
        friend int addFive(Distance);  
    public:  
        Distance() : meter(0) {}  
};  
// friend function Declaration definition  
int addFive(Distance d) {  
    d.meter += 5;  
    return d.meter;  
}  
int main() {  
    Distance D;  
    cout << "Distance : " << addFive(D);  
    return 0;  
}
```

Output :-

Distance : 5

Friend class in C++ :-

A friend class is used when we need to access private and protected members of the class in which it has been declared as a friend. It is also possible to declare only one member function of another class to be a friend.

Ex:- // C++ program to demonstrate the working of friend class

```
#include <iostream>
using namespace std;
// forward declaration
class classB;

class classA {
    private:
        int numA;
        // friend class declaration
        friend class classB;
    public:
        // constructor to initialize numA to 12
        classA() : numA(12) {}
};
```

```
class classB {
    private:
        int numB;
    public:
        classB() : numB(1) {}
        int add() {
            classA objectA;
            return objectA.numA + numB;
        }
};
```

```
int main()
{
    classB objectB;
    cout << "Sum: " << objectB.add();
    return 0;
}
```

Output :-

Sum: 13

Q4) Demonstration with an example, How an operator is overloaded in C++.

Ans)

Operator Overloading:

- * Operator overloading provides ability to an operator to perform other manipulation operations other than its original operation.
- * It preserves the original function of the operator and work with user defined types only.
- * Operator overloading function can be used as a member function as well as friend function.

Types of overloading approaches.

- i) Overloading binary operator.
- ii) Overloading unary operator.
- iii) Overloading binary operator using a friend function.

Ex:-

```
#include <iostream>
using namespace std;

class complexnumber {
    Private:
        int real;
        int imaginary;
    Public:
        complexnumber (int real, int imaginary) {
            this->real = real;
            this->imaginary = imaginary;
        }
        void Print () {
            cout << real << " + i " << imaginary;
        }
        complexnumber operator + (complexnumber c2) {
            complexnumber c3(0,0);
            c3.real = this->real + c2.real;
            c3.imaginary = this->imaginary + c2.imaginary;
            return c3;
        }
};
```

```

int main()
{
    complexnumber c1(3,5);
    complexnumber c2(2,4);
    complexnumber c3=c1+c2;
    c3.print();
    return 0;
}

```

Output :-
5 + i9

Ex} // Binary operator overloading using a Friend Function

```

#include <iostream>
using namespace std;

class A {
public:
    int p, q;
    A()
    {
        this->p = 0;
        this->q = 0;
    }
    A(int r, int s)
    {
        this->p = r;
        this->q = s;
    }
    friend A operator+(A a1, A a2);
};

A operator+(A a1, A a2)
{
    A a3;
    a3.p = a1.p + a2.p;
    a3.p = a1.p + a2.q;
    a3.q = a1.q + a2.p;
    return a3;
}

```

```

int main()
{
    A a1(8,9);
    A a2(10,2);
    A a3;
    a3 = a1 + a2;
    cout << "P and Q values are :- " << a3.P << " " << a3.Q;
    return 0;
}

```

Output :-

P and Q values are :- 18 11

Ex 1 // Unary operator overloading

```

#include <iostream>
using namespace std;

```

```

class A
{

```

```

    public:

```

```

        int P, Q;

```

```

        A(int r, int s)
        {

```

```

            this->P = r;

```

```

            this->Q = s;
        }

```

```

        void operator--()
        {

```

```

            P--;

```

```

            Q--;

```

```

            cout << "P, Q are :- " << P << " " << Q;
        }
    }

```

```

int main() {

```

```

    A a1(7, 5);

```

```

    --a1;

```

```

    return 0;
}

```

Output :-

P, Q are :- 6 4

Q) 5) Illustrate about new and delete operators with an example program.

Ans:- In C++, Deallocation and allocation of memory can be done by using new and delete.

- * The new operator is used to allocate memory to a variable, arrays, objects, etc., where the memory will remain allocated until you manually de-allocate it.
- * It returns the pointer to the newly allocated memory.
- * If using new int; the allocation occurs on a region of memory called the heap.

Syntax:- <data Type PointerName> = new <data Type>

Ex 1

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int *A = new int(95);
    cout << *A << endl;
}
```

Output:-
95

- * Delete operator deallocates the memory that was previously allocated using new.
- * The deallocated memory can be used by operating system for other purposes.

Syntax:- delete pointerVariable

Ex 2

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int *B = new int(18);
    cout << *B << endl;
    delete B;
}
```

O/p:-
18

```

Ex:- #include <iostream>
      using namespace std;

      class emp {
      private
        int eno;
      public:
        emp() : eno(99) {}
        void geteno() {
          cout << "E. No. = " << eno << endl;
        }
      };

      int main()
      {
        emp* p = new emp();
        p->geteno();
        delete p;
        return 0;
      }

```

Output ↓

E. No. = 99

Q) 6) write about Generic programming and explain about function templates and class templates.

Sol ↓ Generic programming allows us to write a general algorithm (Function or class) which will work with all datatypes. using generic programming we achieve

- ① code reusability.
- ② we can avoid function overloading / multiple functions.
- ③ Once a generic algorithm is written, it can be used multiple times.
- ④ Generic programming can be implemented in c++ using templates.
- ⑤ A template is a modal which works for any datatype.

There are two types of functions

- ① Function templates
- ② class templates.

Function Template :

- * Generic functions use the concept of a function template. Generic functions define a set of operations that can be applied to the various types of data.
- * The type of the data that the function will operate on depends on the type of data passed as a parameter.

format:-

template < typename identifier > function_declaration;

Ex :- #include <iostream>
using namespace std;
template < typename T >
T MAX(T x, T y)
{ return (x > y) ? x : y;
}

```
int main()  
{  
    cout << MAX<int>(31, 70) << endl;  
    cout << MAX<double>(3.9, 8.701) << endl;  
    cout << MAX<char>('g', 'e') << endl;  
    return 0;  
}
```

output :-

70
8.701
g

Class Template

class Template can also be defined similarly to the Function Template. when a class uses the concept of Template, then the class is known as generic class.

```
Ex:- #include <iostream>
      using namespace std;

      template < class T, class U >
      class A {
          T x;
          U y;
      public:
          A()
          {
              cout << "constructor called" << endl;
          }
      };

      int main()
      {
          A <char, char> a;
          A <int, double> b;
          return 0;
      }
```

Output:-

constructor called
constructor called